

# An open source IoT edge-computing system for monitoring energy consumption in buildings

Daniel Alfonso Verde Romero, Efrain Villalvazo Laureano<sup>\*</sup>, Ramón Octavio Jiménez Betancourt, Ernesto Navarro Álvarez

Faculty of Electromechanical Engineering, University of Colima, Manzanillo, Colima, Mexico

## ARTICLE INFO

### Keywords:

Edge computing  
Fog computing  
MQTT  
Networks  
Sensors  
Database

## ABSTRACT

This paper presents "An Open IoT Edge Computing System for Monitoring Energy Consumption in Buildings." Implemented at the Faculty of Electromechanical Engineering of the University of Colima, this innovative platform optimizes the pre-existing network infrastructure. Backed by the emerging edge and fog computing paradigms, the solution prioritizes the security of sensitive electrical data, providing detailed power, voltage, and current measurements. Designed to be compatible with various IoT technologies, the research includes practical results that support and validate its effectiveness, highlighting the benefits of edge/fog computing compared to purely cloud-based architectures. It also highlights the versatility of the platform to monitor various variables, emphasizing its applicability in different contexts and monitoring needs.

## 1. Introduction

Monitoring systems involve a wide range of interconnected devices, generating a large volume of data that poses latency and response time challenges for timely decision-making. The traditional centralized processing method, which involves sending large amounts of collected data to a central server in the cloud for processing and transmission, can cause bandwidth bottlenecks and network delays, as mentioned in Ref. [1]. On the other hand, in Ref. [2], edge computing is presented as a solution that optimizes the latency and efficiency of local networks, accelerating processing and facilitating the advancement of deep learning. This technology becomes essential when the volume of data generated exceeds the transfer capacity to a central platform in the cloud or when it is required to minimize latency times to the maximum in specific applications. This technological advance will promote the development of countless applications and services that require exceptionally low latency and high quality of service, as detailed in Ref. [3].

In [4], edge computing is defined as a novel concept in computing. This approach brings cloud computing services and utilities directly to the end user, characterized by fast processing and short application response times.

In [5], it is stated that the first mobile Edge Computing platform, known as RACS (Radio Applications Cloud Server.), was presented by Nokia and IBM in 2013. Despite this milestone, the foundations of Edge

Computing date back to 1990, when content delivery networks (CDNs) were proposed as a solution to improve web performance.

The expansion of the Internet of Things (IoT) and the increasing adoption of 4G/5G networks are transforming how people access and process data, posing challenges to cloud computing capabilities. Edge computing represents a new computing paradigm [6], which reduces the load on IoT device networks by processing data at edge nodes instead of sending it directly to servers in the cloud. In this approach, edge nodes handle intermediate data instead of input data [7].

In the industrial domain, the Industrial Internet of Things (IIoT) emerges strongly, establishing connections between devices through advanced communication technologies, which significantly raises the efficiency and performance of the industry. The application of edge computing technology, detailed in Ref. [8], highlights the ability of IIoT to process, in real-time and at the edge of the network, a substantial portion of sensing data. This strategy effectively addresses the fundamental challenge associated with bandwidth in transmission and latency in decision-making from cloud platforms, all without compromising information security.

Cisco estimated that, by 2022, IoT networks would have connected approximately 50 billion devices. Although the cloud offered solutions for storing and processing data, it presented significant challenges in terms of bandwidth and latency. In addressing these limitations and focusing on latency within the IoT context, Cisco introduced the concept

<sup>\*</sup> Corresponding author.

E-mail address: [villalvazo@ucol.mx](mailto:villalvazo@ucol.mx) (E. Villalvazo Laureano).

of Fog Computing [9]. This groundbreaking architecture, also known as fog computing, provided comprehensive computing, storage, control, and network connectivity capabilities, bringing to life the vision of the Internet of Things [10]. Furthermore, as emphasized in Ref. [11], it became essential for various services characterized by sensitivity to latency, geographical dispersion of devices, mobility, and numerous nodes.

### 1.1. Motivation

The work presented in this paper arose after much research on the Internet of Things. In these works, free platforms were always needed to process and store data in the cloud. Generally, the amount of data allowed in free form is limited; moreover, sensitive data of some users or valuable information from research projects could be put at risk.

Subsequently, the infrastructure available at the University of Colima was analyzed for its security and robustness, and it was concluded that it was appropriate for integrating systems with edge and fog computing. It was also determined that it was possible to carry out data management projects with various devices compatible with the existing infrastructure with a minimum investment and proceeded to carry out this project.

### 1.2. Contributions

An open-source IoT edge computing system designed to be used and modified by other researchers is presented. The system demonstrates the advantages of an edge-computing architecture over a pure cloud in monitoring energy consumption, power quality, water consumption, and temperature. These advantages include lower latency, increased privacy/security, and continuous operation. In addition, it leverages the university's existing computational infrastructure, thus optimizing the resources available to academic institutions.

Thanks to its open and expandable nature, the system allows for efficient scalability by easily integrating additional sensors and applications. The time series data is stored in an SQLite database for future analysis, providing a valuable dataset for applying machine learning methods.

Standard open-source technologies, such as MQTT and Node-RED, facilitate system adoption and modification. This study includes developed measurement and monitoring algorithms, available on this GitHub profile, promoting transparency and knowledge sharing. The presence of these algorithms on GitHub is intended to facilitate access and review by other researchers, encouraging the possibility of additional contributions to improve and expand the measurement and monitoring capabilities on this platform. This paper provides a documented example of an IoT edge computing system for building energy monitoring, with the potential to benefit future research in this area.

### 1.3. Related work

Due to the inherent advantages of edge and fog computing, applications have emerged in various areas, such as clean energy management systems, to drive the evolution of intelligent buildings through IoT. These applications focus on energy saving and efficient energy management. For example, in Ref. [12], an IoT-based energy management system design supported by edge computing infrastructure targeting an intelligent city is proposed. This system employs a deep learning network to improve energy efficiency. Similarly [13], presents an architecture that integrates three systems: monitoring, metering, and a microgrid, leveraging edge computing and an algorithm designed to foster energy savings. On the other hand, the study described in Ref. [14] adopts a cooperative approach that allows edge computing to manage renewable energy's complete and efficient use.

In [15], a proposal that uses an Edge Computing platform in conjunction with IoT to implement an intelligent energy efficiency

system in a public building is presented. The results obtained evidence of several benefits derived from the application of Edge Computing, such as the decrease of the load in transfers to the Cloud and the reduction of costs and network resources. In addition, Edge Computing finds another valuable application in accurately identifying industrial equipment, leading to a significant reduction in the costs associated with manual labeling, as detailed in Ref. [16].

In [17], an edge-computing architecture designed for Industrial Internet of Things (IIoT) applications is presented. This framework demonstrates a task self-organization mechanism with remarkable benefits regarding industrial agility and bandwidth optimization. In addition to the computational advantages inherent to the cloud, such as fast response and network security, integrating resources at the network's edge will significantly improve the quality of service in industrial use cases, especially in IoT-based manufacturing.

In [18], the concept of intelligent cooperative computing in IoT networks is introduced, which combines artificial intelligence (AI) with edge computing. In this approach, selective management of data generated by IoT devices is performed: private user data is stored locally on the devices, while public data is directed to the cloud.

Smart manufacturing requires the support of information technology, data management, and operations, with a particular emphasis on edge computing. As illustrated in the example in Ref. [19], the design of an intelligent manufacturing system incorporating Edge Computing and Blockchain technology demonstrated its effectiveness. In this case, edge computing was used to evenly distribute the computational workload and provide more agile responses to terminal devices.

In this study, scheduling is proposed to improve the energy consumption efficiency of vehicles enabled by mobile edge computing (MEC). A heuristic algorithm is implemented that jointly considers the tasks between the servers and the downlink power consumption of the solar vehicle charging units, as detailed in Ref. [20]. Furthermore, in the es-study presented in Ref. [21], an analysis of a 3D application requiring high resource consumption is carried out. The response delay is evaluated in various scenarios, and it is found that edge computing is essential to meet the latency requirements of virtual and augmented reality-related applications.

As part of the research described in Ref. [22], a tool condition monitoring system has been developed and implemented on a numerical control lathe. This system is supported by an Internet of Things (IoT) platform and employs low-cost sensors with edge computing capabilities.

Fog computing has emerged as a solution for processing real-time data streams with low latency and ample resource capacity, complementing edge and cloud computing. Cisco was one of the pioneers in implementing the Fog computing model, which brings the cloud infrastructure closer to the user's device. This technology is highly beneficial for various industrial applications due to its instant responsiveness. In addition, it offers a cost-effective, flexible, and portable implementation in terms of both hardware and software [23].

Recent research [24] proposes a "toolbox" to strengthen the integrity, security, and privacy of critical IoT infrastructure based on SCADA at the fog layer. This approach focuses on creating a more secure and reliable infrastructure to counter cyber threats affecting the critical infrastructure of the next generation of industrial control systems. In another paper [25], a hierarchical reference architecture for intelligent manufacturing is introduced by implementing edge and fog computing.

A distributed fog computing system for home automation is introduced to facilitate seamless communication between ZigBee and WiFi devices. This system utilizes the capabilities of open-source software, overcoming the limitations of traditional home automation solutions that often rely on expensive central controllers. As emphasized in Ref. [26], fog services demonstrate faster response times than cloud services.

In one study, a MATLAB/Simulink-based model modification is presented to enable real-time communication with the open-source IoT

platform ThingSpeak, which is used in cloud computing. The fog computing layer is modeled using MATLAB function blocks, enabling real-time Smart Grid monitoring [27]. In another research, a distributed mobile smartphone-based fog computing system is described to collect real-time blood glucose data and send it to a remote cloud [28].

An edge and fog computing platform has been developed in Ref. [29] to integrate Bluetooth vehicle identifiers and radar counters for seamless mobility of complex sensors. In addition, an efficient data fusion strategy is implemented.

Fog computing plays a crucial role in agriculture by providing robust support in processing large volumes of data generated by IoT devices embedded in tractors, drones, apps, machinery, and users. Edge computing technology significantly impacts various agricultural applications, such as pest identification, agricultural product safety traceability, autonomous farm machinery, agricultural technology promotion, and intelligent management. In addition, edge computing is narrowing the path toward the evolution to 5G, bringing cloud capabilities closer to end users [30]. On the other hand, in a further approach, an edge computing technology is presented to manage heterogeneous data collection, analysis, prediction, and sensing in strawberry farming; this is achieved through an IoT platform integrating various monitoring services [31].

A system that measures all relevant meteorological variables and directly acquires photovoltaic generation data is proposed for a photovoltaic plant [32]. Designed with open architecture software, it connects to the internet and wirelessly stores the data using fog computing and in the cloud.

In [33], a method for designing intelligent services based on the paradigm of edge computing and fog computing is proposed. It employs an experimental architecture based on embedded devices, which are applied to energy management, security systems, climate control, and information services.

As can be seen, in recent years, many works have been developed with edge computing in the fog, decreasing the problem of latency and data congestion in the cloud and increasing the security of critical data.

## 2. Proposed IoT edge computing platform

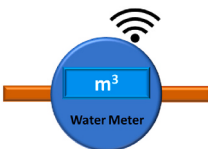
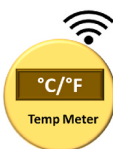
The characteristics of the variables monitored in buildings arise from dynamic processes with various time scales. In addition, energy sensors can generate data on energy consumption every minute, whereas, for power quality analysis, measurements at the millisecond level are required. In contrast, water consumption and temperature sensors provide data in minute intervals. Due to these differences, the same measurement nodes may be equipped with different communication technologies, processing speeds, and sampling rates. For an overview of the generated variables and the associated data volume, see Table 1.

Table 1 presents a comprehensive representation of the diversity of the monitoring platform, covering variables for different purposes, such as water consumption, electricity consumption, energy quality, and environmental variables, such as temperature. It provides an overview of the variables generated and the associated data volume, highlighting the platform's versatility in capturing information for different aspects.

The ability to manage variables with different time scales is highlighted, making the platform adaptable to various monitoring needs in building environments. This comprehensive approach strengthens the platform's ability to address various variables, ensuring its effectiveness and relevance in different monitoring contexts.

Fig. 1 shows the structure of the proposed platform, which follows a typical architecture in edge computing systems but stands out for its broad compatibility with various development boards and communication protocols. In the lower layer are the assets and sensors in charge of collecting data on the electrical variables under monitoring. The embedded board is a gateway for preprocessing and communicating the data via the MQTT protocol. The edge level is the middle layer, with the server hosting the MQTT broker, the Node-RED tool for visualization,

**Table 1**  
Variables monitored on the platform.

| Sensor                                                                             | Variables              | Data type | Sampling frequency (s) | KB/day   |
|------------------------------------------------------------------------------------|------------------------|-----------|------------------------|----------|
|   | Vrms<br>Irms<br>PF kWh | Float     | 5 min                  | 5 KB     |
|   | V<br>I                 | Float     | 1 min                  | 115.2 KB |
|  | Q<br>M <sup>3</sup>    | Float     | 5 min                  | 2.5 KB   |
|  | Temp<br>Humidity       | Float     | 5 min                  | 2.5 KB   |

In this context, a proposed platform follows a typical architecture but stands out for its high compatibility with different development boards and communication protocols.

and the SQLite database for persistent storage. Finally, at the top layer is the cloud, allowing remote access to dashboards and reports. This tiered segmentation provides benefits such as low latency, increased security, and the ability to operate in isolation in case of internet connectivity failure.

The main hardware features of each level are described below.

### 2.1. Edge level hardware

Deploying real-time applications requires data processing and storage close to end devices and end users. In this context, the hardware infrastructure at the edge plays a crucial role in enabling efficient and scalable computing. This section overviews the various hardware components used at the edge level, including processors, memory, storage, and communications. For experimental purposes, the sensor nodes have been configured to transmit data every 10 s, which fits the needs of the electrical variable monitoring application. It is estimated that the volume of data stored during 24 h will be approximately 3 megabytes, equivalent to the information of 5 power monitoring variables.

To achieve this goal, hardware at the edge level must be able to efficiently and reliably handle data-intensive workloads. The following are the critical hardware components at the edge level that are used because of their characteristics that meet the requirements for experimentation. A computer equipped with a high-performance 7th generation Intel i7 processor with multiple cores, 8 GB RAM, and a 1 TB hard disk for storage is used. Devices facilitating communication between the platform and the sensors include a LAN switch for server communication on the local network and a Wi-Fi access point to provide access to clients

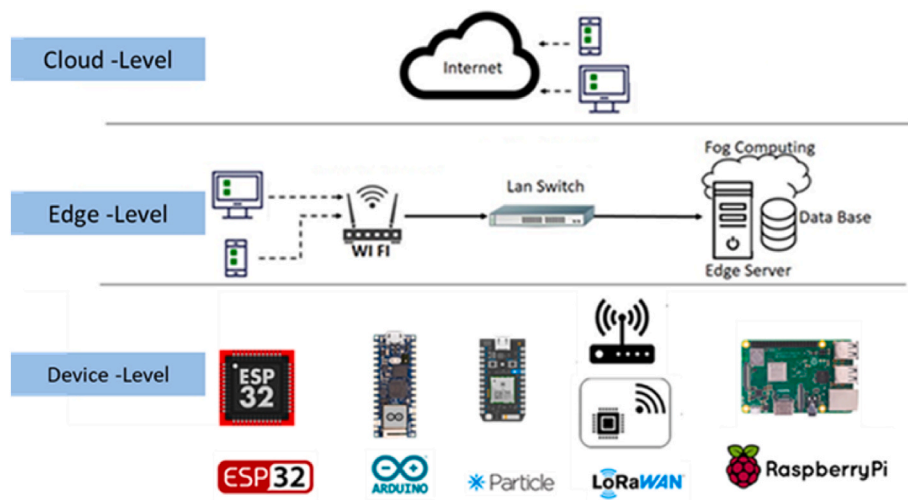


Fig. 1. Structure of the proposed platform.

wishing to monitor. These components are described in detail in Table 2.

2.2. Software on the server at the edge level

Linux, as an operating system (OS), is recognized as a highly efficient open-source platform, ideal for interacting with open-source software tools in developing applications for the Internet of Things (IoT). For this reason, the Ubuntu 18.04.2 LTS distribution was chosen as the operating system on the Edge server. On this basis, the following essential software tools were installed for the creation of the monitoring application: a) Eclipse Mosquitto, an MQTT broker that offers support for Message Queuing Telemetry Transport (MQTT) protocol versions 5.0, 3.1.1 and 3.1, b) Node-RED, a programming tool that simplifies the interconnection of hardware devices, APIs and online services, speeding up the monitoring process and c) SQLite, a C language library that implements a SQL database engine. Fig. 2 visually illustrates the interaction of these software tools on the server.

MQTT is a lightweight protocol created by IBM, designed to simplify machine-to-machine (M2M) communications, especially when devices face bandwidth and power consumption constraints. This protocol is based on four essential elements: the broker, the publishing/subscription model, topics, and quality of service (QoS).

Despite the variety of brokers available in the market, such as HiveMQ, Mos-ca/Aedes, and Azure IoT, this project decided to use the MQTT Mosquitto Eclipse broker. This choice was based on its open-source EPL/EDL license and its no-cost availability, which makes it a suitable choice for installation in the Edge Computing server.

2.3. Embedded card and asset level sensors

The platform supports various development boards, including ESP32, Arduino, Raspberry Pi, Particle Photon, and others supporting MQTT protocol configuration. In addition, it offers support for various communication protocols, such as Wi-Fi, LoRaWAN, Ethernet, and USB, which simplifies integration with other IoT systems and devices. The platform is highly customizable and scalable, making it suitable for incorporating third-party sensors, devices, or software and meeting

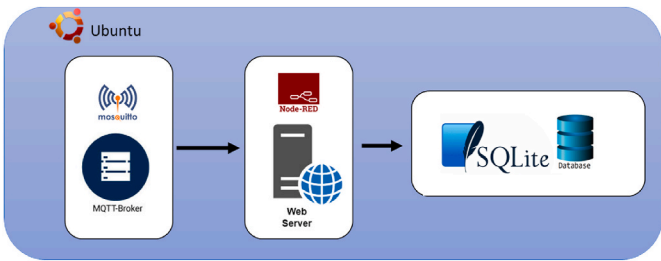


Fig. 2. Software on the edge server.

various requirements and configurations.

For this IoT project, Particle's Photon embedded development platform was implemented. This platform has a 120 MHz ARM Cortex M3 microcontroller, a 1 MB flash memory, 802.11 b/g/n Wi-Fi connectivity, and an integrated antenna. The sensors used to measure electrical parameters, voltage, and current are connected to the analog input pins A0 and A1, two of the channels of the 12-bit analog-to-digital (A/D) converter (0–4095).

Fig. 3 presents the essential components used in the project, the complete electronic schematic, and the final assembly of the integrated board.

As mentioned above in Fig. 4, the IoT device implemented in this project is installed in the main electrical distribution board of the Cisco laboratory of the Faculty of Electromechanical Engineering of the University of Colima. Fig. 6 shows the mounting on the panel, where four wires are connected: one pair of terminals for voltage measurement and two to the hook sensor that registers current readings of one of the phases.

The algorithm for measuring the development board is available in the Git Hub repository. <https://github.com/verdebit/Photon-Particle-Edge-Computing.git>.

2.4. Implementing the MQTT Client-Server

The MQTT IoT client, responsible for sending the messages generated by the sensors, must establish a connection with the MQTT broker. It uses an MQTT client library, which specifies the host server address, port 1883, and the user name and password according to the MQTT server configuration. The publication of the variables is carried out by assigning them to a specific MQTT topic. The publication is based on a publish-subscribe model, in which a device broadcasts messages on a

Table 2  
Edge-level hardware components.

| Component       | Feature             |
|-----------------|---------------------|
| Computer        | HP PRODEX CORE i7   |
| Lan switch      | Cisco Catalyst 2950 |
| Aces Point Wifi | Huawey W55200       |



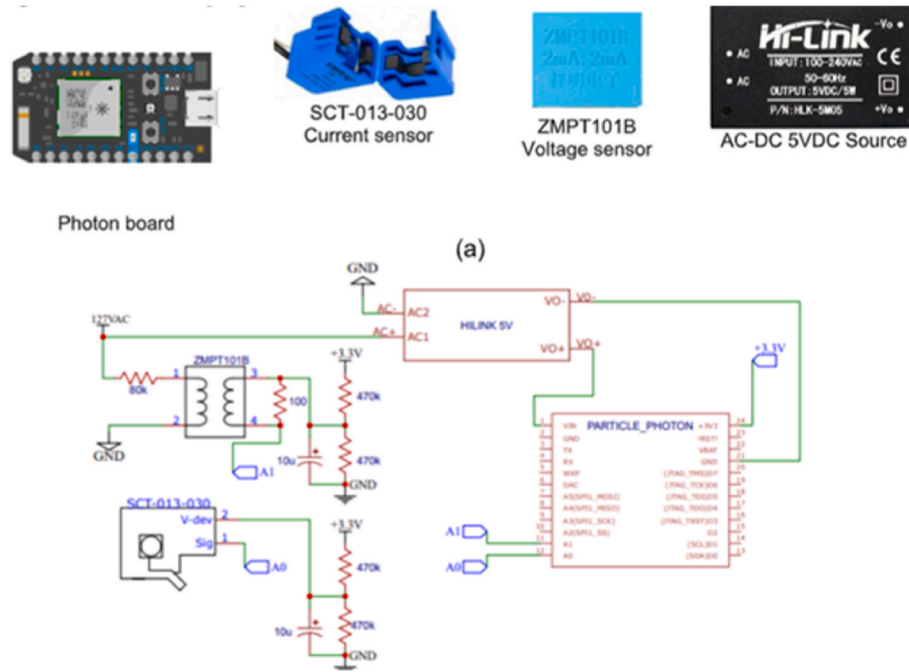


Fig. 3. IoT measuring card.

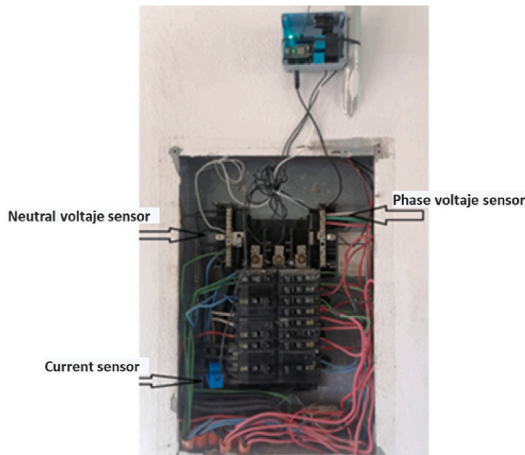


Fig. 4. The IoT meter on the main board.

particular topic. In contrast, other devices subscribe to that topic to receive the corresponding messages. In this context, the web server implemented in Node-RED subscribes to a specific topic on localhost, as illustrated in Fig. 5.

Para habilitar la funcionalidad MQTT en la tarjeta, se procede a instalar la biblioteca MQTT siguiendo el procedimiento detallado en la referencia [32]. En el dispositivo medidor IoT implementado, se incorpora la biblioteca MQTT de Particle mediante el instalador de librerías.

Para lograr esto, se accede al enlace <https://build.particle.io/build> a través de un navegador web. Tras autenticarse con las credenciales de usuario, se despliega la consola de Particle y se selecciona "web IDE". Una vez allí, se accede a la sección de librerías y se localiza la librería MQTT, como se ejemplifica en la Figura 5.

The MQTT library is installed following the procedure detailed in Ref. [30] to enable MQTT functionality on the card. In the implemented IoT meter device, the Particle MQTT library is incorporated using the library installer. To achieve this, navigate to <https://build.particle.io/build> through a web browser. After authenticating with user credentials, the Particle console is displayed, and "web IDE" is selected. Once there, go to the libraries section and locate the MQTT library, as illustrated in Fig. 6.

The MQTT library is imported into the project in the Particle Web-IDE and click on the "Add to project" button to import it.

The code is used to create a connection instance of the MQTT client to the server and to configure the server address and port Fig. 7.

Before sending or receiving messages through MQTT, the MQTT client is connected to the server. This connection is established using the connect method of the client.connect instance. Subsequently, the variables are published in a specific topic defined using the client.publish method of the same instance. This process is illustrated in Fig. 8.

## 2.5. Data storage in SQLite

To implement the database on the server, we proceeded to install SQLite following the specific instructions for the Linux operating system.

Once SQLite was installed, a new database and a new table were

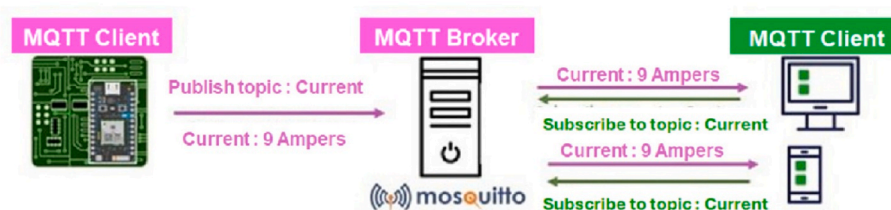


Fig. 5. Publishing and subscribing to MQTT Client-Server messages.



Figura 6. Installation of libraries.

```

1 #include <time.h>
2 #include "math.h"
3 #include "stdio.h"
4 #include <MQTT.h>
5 SYSTEM_THREAD(ENABLED);
6 void callback(char* topic, byte* payload, unsigned int length);
7 const uint8_t server_TESIS[] = { 148, 213, 117, 98};
8 MQTT client(server_TESIS, 1883, callback);
9
10 void callback(char* topic, byte* payload, unsigned int length) {
11
12 }

```

Fig. 7. MQTT server configuration.

```

113 String V = String(VRMS_m);
114 String I = String(IRMS_a);
115 String PP = String(PPROM);
116 String SFP = String(FP);
117 //String DELTA_P = String(DP);
118 //String DELTA_S = String(DS);
119 String SPA = String(PA);
120
121
122 client.connect("server_TESIS");
123
124
125 // publish/subscribe
126 if (client.isConnected()) {
127   client.publish("VOLTAGE", V);
128   client.publish("CORRIENTE", I);
129   client.publish("PPROMEDIO", PP);
130   client.publish("BD/FACTOR DE POTENCIA", SFP);
131   client.publish("BD/DELTA_P", DELTA_P);
132   client.publish("BD/DELTA_S", DELTA_S);
133   client.publish("PAPARENTE", SPA);
134 }
135

```

Fig. 8. Calling the client method.

created using the litedb node in Node-RED; through the injection of SQL commands using the inject node, this node sends the SQL commands in the form of input msg.topic and receives responses of type msg.payload in the console through the output debug node, this creates the table where they are stored with the columns of the electrical variables that are published in the MQTT broker; in turn, this data was sent with the SQL command to the litedb injection node with the following configuration, see Fig. 9.

The variable records have been incorporated into the table named "BIMETER," with fields including "VOLTAGE," "CURRENT," "AVERAGE POWER," "APPARENT POWER," and "POWER FACTOR." Next, the workflow was executed in Node-RED, and the correct data insertion into the SQLite database was verified. Using DBBrowser for SQLite, the database was visualized, thus confirming the proper insertion of these data into Table 3.

Fig. 10 shows the flowchart that comprehensively presents the process of the energy monitoring system based on edge computing. It starts with capturing raw data from the asset sensors or electricity meters. This

data is then transmitted to the Photon board, where basic preprocessing is performed. The raw data filtered by Photon is sent to the edge server via WiFi, where further correction and filtering are performed.

The MQTT client in Photon displays the processed and raw data in MQTT topics, while Node-RED on the server subscribes to receive that data. These are used for real-time graph generation on the web dashboard and are simultaneously stored in the SQLite database. Subsequently, a comprehensive analysis of the stored data is carried out, culminating in generating reports and visualizations. This diagram shows exhaustively the flow of information throughout the different stages of the monitoring system implemented by edge computing.

### 3. Results

The proposed system was tested using two IoT devices of different brands and characteristics. The first one was installed in the load center of the telephony laboratory, monitoring electrical voltage and current variables in real-time. Then, the second development board was used to monitor the ambient temperature. The most significant results were obtained in real-time energy monitoring, described below.

Experimentation and testing on the proposed IoT-Edge Computing platform are oriented toward monitoring electrical parameters. This approach involves a careful selection of hardware and software suitable for interconnection within the internal data network of the University of Colima. It is approached by developing embedded software designed to accurately capture the parameters reported by the sensors and efficiently publish them in the corresponding server. In addition, the development of a web interface for continuous monitoring, the implementation of storage solutions to facilitate data visualization and analysis, the integration of security measures, and a debugging process.

Fig. 11 shows the general diagram of the proposed ecosystem. It is essential to accurately define the requirements, including the electrical parameters to be monitored, the type of connectivity, the necessary bandwidth, the data storage mode, and the security requirements. In addition, Fig. 11 illustrates the different levels of data management. Initially, essential data is retained at the Edge layer, followed by identifying and storing relevant information at the Cloud layer. This layer includes both values that could destabilize the system and alerts that require attention beyond the university network. Finally, the instrumentation required to measure electrical parameters and system loads is placed in the Assent phase.

#### 3.1. Real-time data visualization

Implementing a data dashboard provides a tool that enables clear

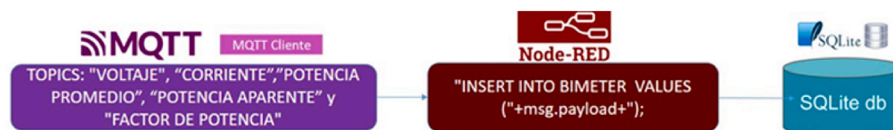


Fig. 9. Inserting variables into the SQLite database.

**Table 3**  
Bimeter inspection of data stored in SQLite DB.

| INDEX | VOLTAGE    | CURRENT   | AVERAGE POWER | APPARENT POWER | POWER FACTOR |
|-------|------------|-----------|---------------|----------------|--------------|
| 1     | 115.610825 | 12.41246  | 1379.561768   | 1433.954102    | 0.961357     |
| 2     | 115.578781 | 12.403416 | 1376.747803   | 1439.045044    | 0.960362     |
| 3     | 115.537575 | 12.381578 | 1373.200073   | 1433.571655    | 0.959919     |
| 4     | 115.840179 | 11.997768 | 1306.725342   | 1380.780029    | 0.94021      |
| 5     | 114.963036 | 21.414545 | 2437.603027   | 2439.38208     | 0.990138     |
| 6     | 115.685608 | 12.439275 | 1381.205078   | 1436.980591    | 0.959807     |
| 7     | 115.703171 | 12.419544 | 1379.526978   | 1435.014771    | 0.960018     |

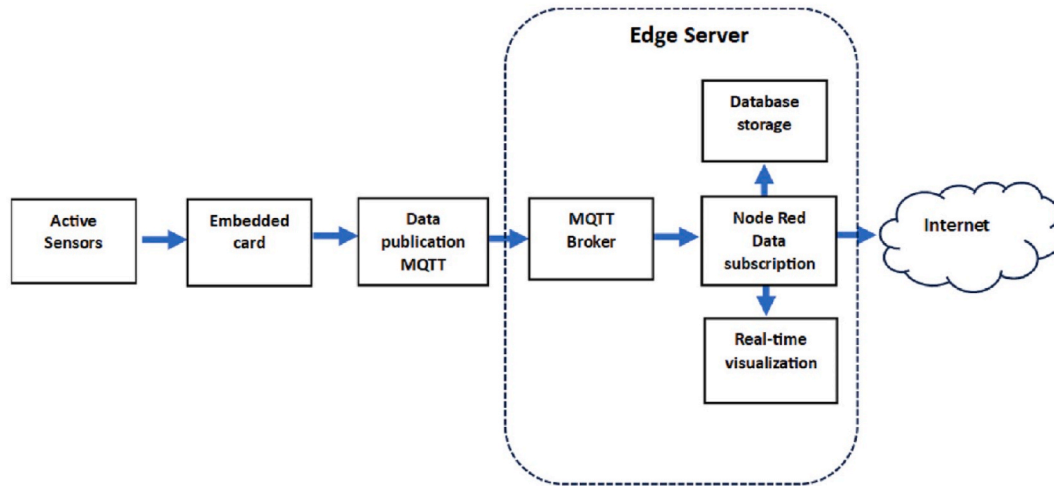


Fig. 10. Process flow diagram.

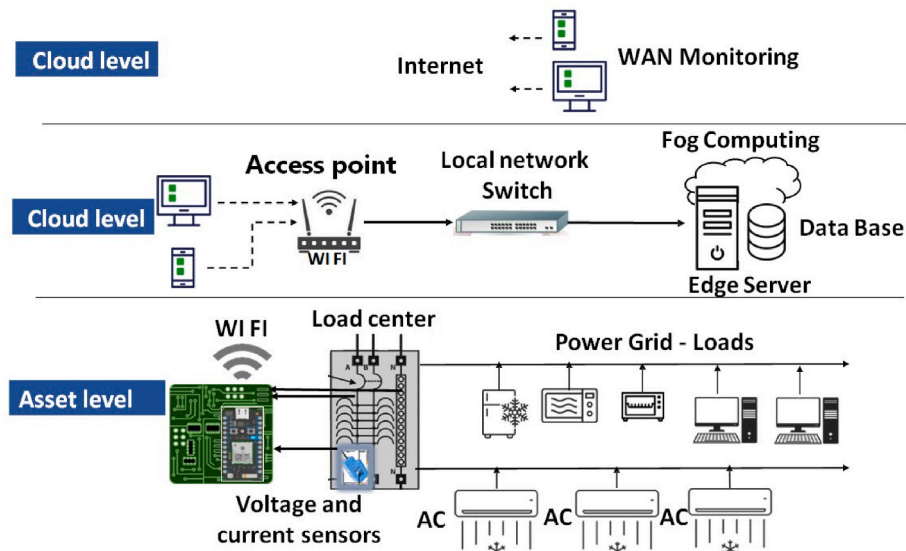


Fig. 11. General diagram of the proposed ecosystem.

and concise visualization of sensor information. Visualization simplifies decision-making and tracking relevant metrics in real time, making it easier to identify patterns and trends. This approach, in turn, contributes to informed decision-making.

In the configured MQTT (Mosquitto-Eclipse) broker, the connection of the MQTT node to the Mosquitto server was established by using the "MQTT" "input" node of No-de-RED. For the configuration of this node, it is necessary to specify the IP address of the broker and the corresponding port. In this case, the configuration includes the IP address of the local

server (localhost), and the port 1883 is used.

After connecting to the broker, the MQTT topics to be monitored ("VOLTAGE," "CURRENT," "AVERAGE POWER," "APPARENT POWER," and "POWER FACTOR") were detailed. The connection was carried out using the "MQTT" node in the "input" tab. In the configuration of this node, the MQTT topic to be monitored is specified. In addition, the possibility of specifying a pattern of MQTT topics using wildcards is provided, thus allowing the monitoring of multiple topics if necessary.

El nodo MQTT fue enlazado an un nodo de función para posibilitar el

procesamiento de los datos recibidos. Se empleó el lenguaje JavaScript para realizar la transformación de los datos antes de ser almacenados en la base de datos BD.

The MQTT node was linked to a function node to enable the processing the received data. JavaScript language was used to transform the data before it was stored in the DB database.

Likewise, the MQTT node was linked to a visualization node for data display. Specifically, a line graph node was employed to depict real-time sensor data, accompanied by the integration a customizable widget to tailor the appearance and functionality of the data visualization.

Once the Node-RED flow was configured to receive and visualize data from the MQTT broker, it was possible to monitor the sensors in real time and analyze the behavior of the variables over 24 h employing the "chart" and "gauge" nodes. Fig. 12 displays the instantaneous values of the measured variables. The monitoring graphs, shown in Figs. 13–17, provide the result of the dashboard. This interaction with the widgets and graphs allows informed decisions to be made based on the data received.

A crucial segment of the acquired results stems from data mining conducted on electrical variables such as voltage, current, active power, apparent power, and power factor. These datasets were systematically stored in an SQLite database within a pre-established table labeled 'BIMETER.' This strategic step aims to streamline forthcoming tasks, encompassing in-depth analysis, statistical assessments, and the application of various machine learning methods. These methodologies will leverage the dataset amassed during experimentation on the dedicated platform designed for measuring electrical variables.

The generated table, with the stored variable fields, is available for verification through the following link. A database file in.csv format with a 24-h recording is provided, accessible here: [https://github.com/verdebit/Photon-Particel\\_Edge-Computing/blob/main/datos\\_25\\_mayo.csv](https://github.com/verdebit/Photon-Particel_Edge-Computing/blob/main/datos_25_mayo.csv). This resource is intended to allow verification of the stored data.

#### 4. Conclusions

The implementation of an edge computing-based energy monitoring system has demonstrated a remarkable reduction in latency compared to the cloud-only approach, thanks to the ability to analyze data near the source.

Edge-level data processing has boosted the efficiency and response speed of the system in real-time, thus providing a framework conducive to informed decision-making.

The strategic combination of edge computing and fog for IoT monitoring has proven effective, capitalizing on local resources without relying exclusively on cloud infrastructure.

The developed platform stands out for its efficiency and positions itself as a scalable and low-cost solution for IoT implementation, optimally leveraging the institution's pre-existing infrastructure.

The choice of open-source software has allowed the solution to be tailored to specific requirements with minimal investment, thus emphasizing the flexibility and customization inherent in this approach.

The implemented algorithms have been shared in an open-source repository, promoting collaboration and continuous improvement within the community.

The system's inherent flexibility allows easy expansion for incorporating new sensors, devices, and applications, adapting to evolving

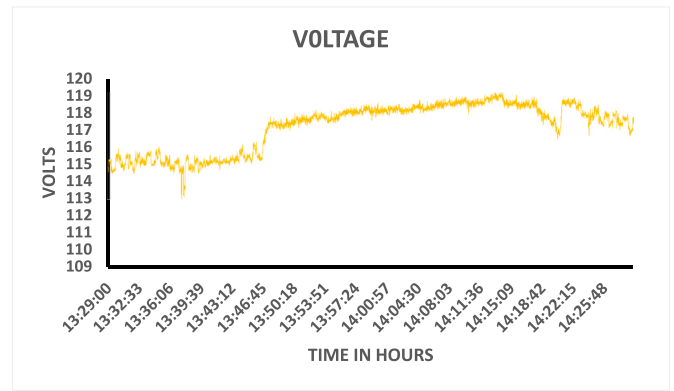


Fig. 13. Voltage.

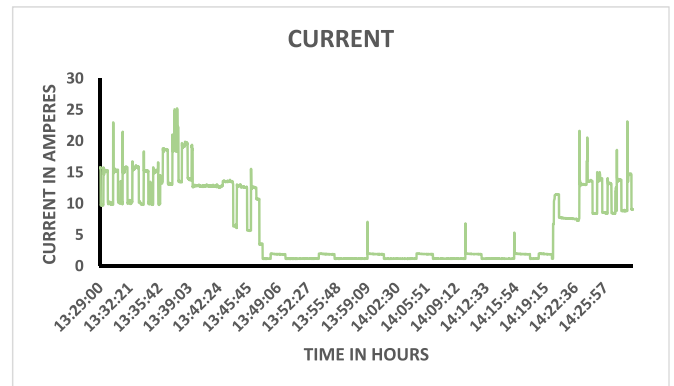


Fig. 14. Current.

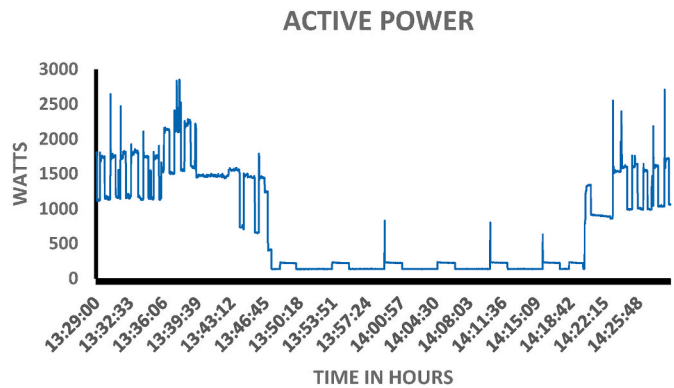


Fig. 15. Active power.

needs.

A comprehensive real-time energy monitoring solution has been successfully achieved, standing out for its scalability, efficiency, and security, thanks to the implementation of edge/fog computing.

It opens up the possibility of exploring artificial intelligence techniques at the network's edge to analyze the collected data, thus

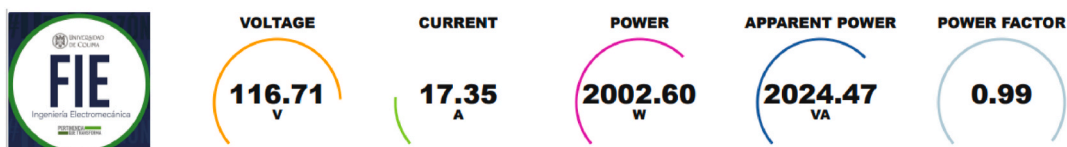


Fig. 12. Dashboard with meters for monitoring variables.



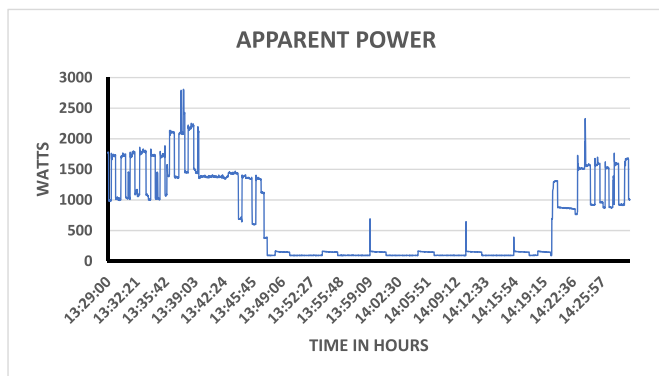


Fig. 16. Apparent power.

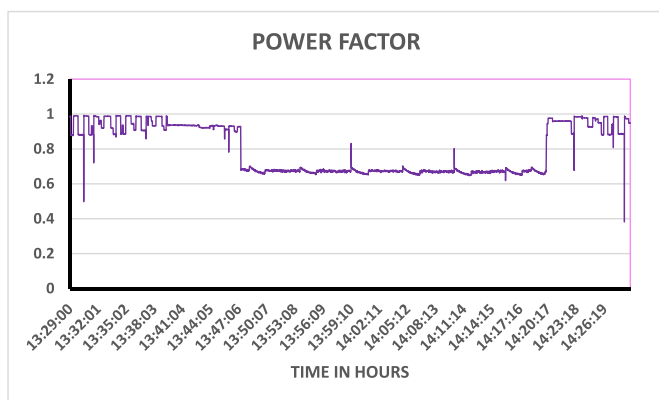


Fig. 17. Power factor.

providing high-level processed information for more informed decision-making.

The presented edge computing-based energy monitoring solution provides a scalable model. It stands out for its low cost, making it a viable option for academic institutions or organizations looking to implement IoT leveraging their existing infrastructure.

#### CRedit authorship contribution statement

**Daniel Alfonso Verde Romero:** Formal analysis, Investigation, Software, Supervision, Visualization. **Efrain Villalvazo Laureano:** Conceptualization, Investigation, Methodology, Writing – original draft, Writing – review & editing. **Ramón Octavio Jiménez Betancourt:** Conceptualization, Formal analysis, Supervision, Writing – review & editing. **Ernesto Navarro Álvarez:** Conceptualization, Software, Writing – review & editing.

#### Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

#### Data availability

There is a link in the article that goes directly to the data of the platform in use; if the reviewers require anything additional, it will be provided without any problem.

#### Acknowledgments

We sincerely thank the Universidad de Colima for supporting us with research facilities and laboratories. We are also very grateful to CON-AHCYT for their support with SNI scholarships.

#### References

- [1] H. Ning, Y. Li, F. Shi, L.T. Yang, Heterogeneous edge computing open platforms and tools for internet, *Future Generation Computer Systems* 106 (1) (2020) 67–76.
- [2] F. Wang, M. Zhang, X. Wang, X. Ma, J. Liu, Deep learning for edge computing applications, *IEEE Access* 8 (1) (2020) 58322–58336, <https://doi.org/10.1109/ACCESS.2020.2982411>.
- [3] P. Porambage, J. Okwuibe, M. Liyanage, M. Ylianttila, T. Taleb, Survey on multi-access edge computing for internet of things realization, *IEEE Communications Surveys & Tutorials* 20 (4) (2018) 2961–2991, <https://doi.org/10.1109/COMST.2018.2849509>.
- [4] W.Z. Khan, E. Ahmed, S. Hakak, I. Yaqoob, A. Ahmed, Edge computing: a survey, *Future Generation Computer Systems* 97 (1) (2019) 219–235, <https://doi.org/10.1016/j.future.2019.02.050>.
- [5] I.S.-C. Sittón-Candanedo, R.S. Alonso, J.M. Corchado, S. Rodríguez-González, R. Casado-Vara, A review of edge computing reference architectures and a new global edge proposal, *Future Generation Computer Systems* 99 (1) (2019) 278–294, <https://doi.org/10.1016/j.future.2019.04.016>.
- [6] F. Liu, G. Tang, Y. Li, Z. Cai, X. Zhang, T. Zhou, A survey on edge computing systems and tools, *Proceedings of the IEEE* 107 (8) (2019) 1537–1562, <https://doi.org/10.1109/JPROC.2019.2920341>.
- [7] H. Li, K. Ota, M. Dong, Learning IoT in edge: deep learning for the internet of things with edge computing, *IEEE Network* 32 (1) (2018) 96–101, <https://doi.org/10.1109/MNET.2018.1700202>.
- [8] T. Qiu, J. Chi, X. Zhou, Z. Ning, M. Atiquzzaman, D.O. Wu, Edge computing in industrial internet of things: architecture, advances and challenges, *IEEE Communications Surveys & Tutorials* 22 (4) (2020) 2462–2488, <https://doi.org/10.1109/COMST.2020.3009103>.
- [9] M.A.-k. Al-khafajiy, T. Baker, H. Al-Libawy, Z. Maamar, M. Aloqaily, Y. Jararweh, Improving fog computing performance via fog-2-fog collaboration, *Future Generation Computer Systems* 100 (1) (2019) 266–280, <https://doi.org/10.1016/j.future.2019.05.015>.
- [10] L. Liu, Z. Chang, X. Guo, S. Mao, T. Ristaniemi, Multiobjective optimization for computation offloading in fog computing, *IEEE Internet of Things Journal* 5 (1) (2018) 283–294, <https://doi.org/10.1109/JIOT.2017.2780236>.
- [11] M. Sookhak, F.R. Yu, Y. He, H. Talebian, S.N. Safa, N. Zhao, M.K. Khan, N. Kumar, Augmentation of fog computing using vehicular cloud, *IEEE Vehicular Technology Magazine* 12 (3) (2017) 55–64, <https://doi.org/10.1109/MVT.2017.2667499>.
- [12] Y. Liu, C. Yang, S. Xie, L. Jiang, Y. Zhang, Intelligent edge computing for IoT-based energy management in Smart cities, *IEEE Network* 33 (2) (2019) 111–117, <https://doi.org/10.1109/MNET.2019.1800254>.
- [13] S. Chen, H. Wen, J. Wu, W. Lei, W. Hou, W. Liu, A. Xu, Y. Jjiang, Internet of things based Smart grids supported by intelligent edge computing, *IEEE Access* 7 (1) (2019) 74089–74102, <https://doi.org/10.1109/ACCESS.2019.2920488>.
- [14] Y. Liu, C. Yang, L. Jiang, S. Xie, Y. Zhang, Intelligent edge computing for IoT-based energy management in Smart cities, *IEEE network* 33 (2) (2019) 111–117, <https://doi.org/10.1109/MNET.2019.1800254>.
- [15] I. Sittón-Candanedo, R.S. Alonso, Ó. García, L. Muñoz, S. Rodríguez-González, Edge computing, IoT and social computing in Smart energy scenarios, *Sensors* 19 (15) (2019) 3353, <https://doi.org/10.3390/s19153353>.
- [16] C.F. Lai, W.C. Chien, L.T. Yang, W. Qiang, LSTM and edge computing for big data feature recognition of industrial electrical equipment, *IEEE Transactions on Industrial Informatics* 15 (4) (2019) 2469–2477, <https://doi.org/10.1109/TII.2019.2892818>.
- [17] B. Chen, J. Wan, A. Celesti, D. Li, H. Abbas, Q. Zhang, Edge computing in IoT-based manufacturing, *IEEE Communications Magazine* 56 (9) (2018) 103–109, <https://doi.org/10.1109/MCOM.2018.1701231>.
- [18] C. Gong, F. Lin, X. Gong, Y. Lu, Intelligent cooperative edge computing in internet of things, *IEEE Internet of Things Journal* 7 (10) (2020) 9372–9382, <https://doi.org/10.1109/JIOT.2020.2986015>.
- [19] C.K.M. Lee, Y.Z. Huo, S.Z. Zhang, K.K.H. Ng, Design of a Smart manufacturing system with the application of multi-access edge computing and blockchain technology, *IEEE Access* 8 (1) (2020) 28659–28667, <https://doi.org/10.1109/ACCESS.2020.2972284>.
- [20] Z. Ning, J. Huang, X. Wang, J.J.P.C. Rodrigues, L. Guo, Mobile edge computing-enabled internet of vehicles: toward energy-efficient scheduling, *IEEE Network* 33 (5) (2019) 198–205, <https://doi.org/10.1109/MNET.2019.1800309>.
- [21] G. Premsankar, M.D. Francesco, T. Taleb, Edge computing for the internet of things: a case study, *IEEE Internet of Things Journal* 5 (2) (2018) 1275–1284, <https://doi.org/10.1109/JIOT.2018.2805263>.
- [22] J.M. Failing, J.V. Abellán-Nebo, S. Benavent Nàcher, P. Rosado Castellano, F. Romero Subirón, A tool condition monitoring system based on low-cost sensors and an IoT platform for rapid deployment, *Processes* 11 (3) (2023) 668, <https://doi.org/10.3390/pr11030668>.
- [23] S. Khan, S. Parkinson, Q. Qin, Fog computing security: a review of current applications and security solutions, *Journal of Cloud Computing* 6 (1) (2017) 1–22.
- [24] T. Baker, M. Asim, A. MacDermott, F. Iqbal, F. Kamoun, B. Shah, O. Alfandi, M. Hammoudeh, A secure fog-based platform for SCADA-based IoT critical

- infrastructure, *Software: Practice and Experience* 50 (5) (2020) 503–518, <https://doi.org/10.1002/spe.2688>.
- [25] Q. Qi, F. Tao, A Smart manufacturing service system based on edge computing, fog computing, and cloud computing, *IEEE Access* 7 (1) (2019) 86769–88677, <https://doi.org/10.1109/ACCESS.2019.2923610>.
- [26] I. Froiz-Míguez, T.M. Fernández-Caramés, P. Fraga-Lamas, L. Castedo, Design, implementation and practical evaluation of an IoT home automation system for fog computing applications based on MQTT and ZigBee-WiFi sensor nodes, *Sensors* 18 (8) (2018) 2660, <https://doi.org/10.3390/s18082660>.
- [27] M. Forcan, M. Maksimović, Cloud-Fog-based approach for Smart Grid monitoring, *Simulation Modelling Practice and Theory* 101 (1) (2020) 101988, <https://doi.org/10.1016/j.simpat.2019.101988>.
- [28] T.M. Fernández-Caramés, I. Froiz-Míguez, O. Blanco-Novoa, P. Fraga-Lamas, Enabling the internet of mobile crowdsourcing health things: a mobile fog computing, blockchain and IoT based continuous glucose monitoring system for diabetes mellitus research and care, *Sensors* 19 (15) (2019) 3319, <https://doi.org/10.3390/s19153319>.
- [29] G. Mujica, R. Rodríguez-Zurrutero, M.R. Wilby, J. Portilla, A.B. Rodríguez González, A. Araujo, T. Riesgo, J.J. Vinagre Díaz, Edge and fog computing platform for data fusion of complex heterogeneous sensors, *Sensors* 18 (11) (2018) 3630, <https://doi.org/10.3390/s18113630>.
- [30] Y. Kalyani, R. Collier, A systematic survey on the role of cloud, fog, and edge computing combination in Smart agriculture, *Sensors* 21 (12) (2021) 5922, <https://doi.org/10.3390/s21175922>.
- [31] M. Cruz, S. Mafra, E. Teixeira, F. Figueiredo, Smart strawberry farming using edge computing and IoT, *Sensors* 22 (15) (2022) 5866, <https://doi.org/10.3390/s22155866>.
- [32] G. Costa Gomes de Melo, I. Cavalcante Torres, Í. Bezzera Queiroz de Araújo, D. Bibiano Brito, E. de Andrade Barboza, A low-cost IoT system for real-time monitoring of climatic variables and photovoltaic generation for Smart Grid application, *Sensors* 21 (9) (2021) 3293, <https://doi.org/10.3390/s210>.
- [33] F.J. Ferrández Pastor, H. Mora, A. Jimeno Morenilla, B. Volckaert, Deployment of IoT edge and fog computing technologies to develop Smart building services, *Sustainability* 10 (11) (2018) 3832, <https://doi.org/10.3390/su10113832>.